



Informatik II – Machine Learning

Informatik II – Woche 2

19. / 20. Mai 2026

warmer Empfehlung: ETH bietet als nur Serien!!

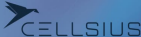
Open Day

21. May 2026, 14.00 Uhr

Get to know the projects and enjoy
FREE BEER & BBQ at the ETH Hangar!

Exclusively for students

Sign Up



Heutiges Programm

Neural Networks

Programmierübung

Training

Programmierübung

Entscheidungsbäume

Programmierübungen

1. Neural Networks

Das Problem: Lineare Modelle sind nicht genug

Lineare Regression und logistische Regression können nur **lineare** Entscheidungsgrenzen lernen.

recall:

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \dots + w_D x_D$$

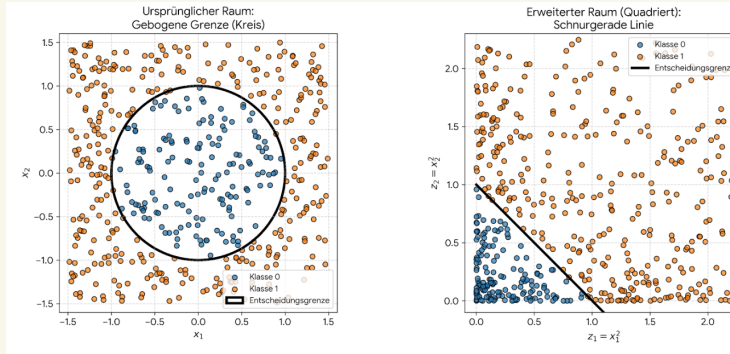
D Merkmale: jedes w_j ist das Gewicht des Merkmals x_j .

„Linear“ bezieht sich auf die Gewichte, *nicht* auf die Eingabe.

Gekrümmte Daten? Einfach neue Merkmale hinzufügen:

$$\hat{y} = w_0 + w_1 x + w_2 x^2$$

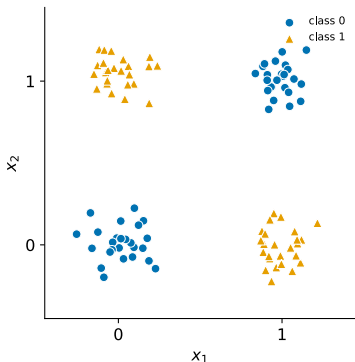
beispiel mit quadratischen features



Das Problem: Lineare Modelle sind nicht genug

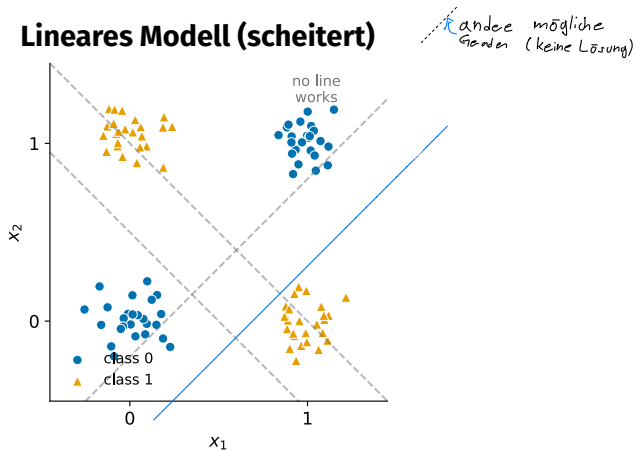
Lineare Regression und logistische Regression können nur **lineare** Entscheidungsgrenzen lernen.

Das XOR-Problem: Wie lässt sich die Entscheidungsgrenze für den folgenden Datensatz vorhersagen?



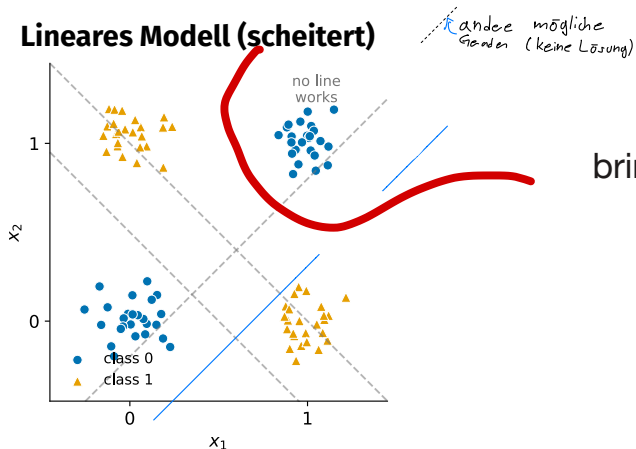
Das Problem und die Lösung: XOR

Lineares Modell (scheitert)



Lineare Regression kann
XOR-Daten nicht mit einer
einzigen Geraden trennen.

Das Problem und die Lösung: XOR

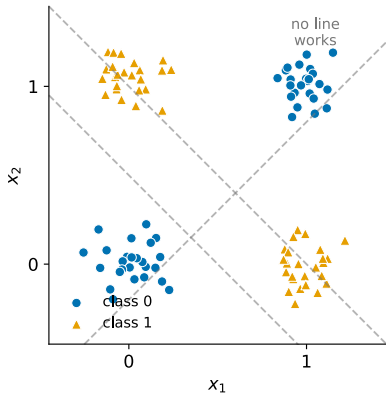


bringt hier auch nichts!

Lineare Regression kann
XOR-Daten nicht mit einer
einzigen Geraden trennen.

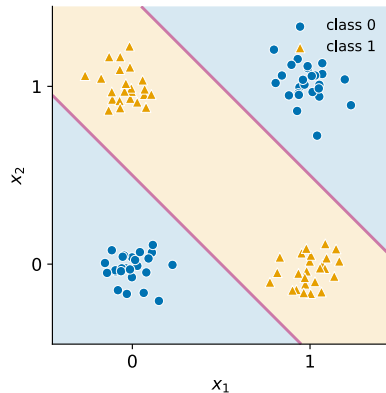
Das Problem und die Lösung: XOR

Lineares Modell (scheitert)



Lineare Regression kann XOR-Daten nicht mit einer einzigen Geraden trennen.

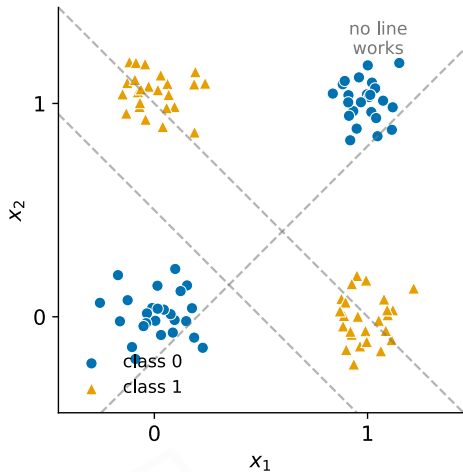
Neuronales Netz (funktioniert!)



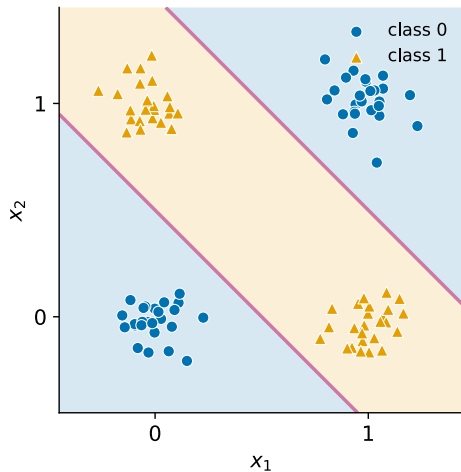
Ein zweischichtiges Netz mit Aktivierungsfunktionen lernt die richtige Grenze.

Das Problem und die Lösung: XOR

Lineares Modell (scheitert)



Neuronales Netz (funktioniert!)



Neural Network Architektur: Schichten und Gewichte

Ein neuronales Netz ist ein Stapel von **Schichten**. Jede Schicht hat:

Neural Network Architektur: Schichten und Gewichte

Ein neuronales Netz ist ein Stapel von **Schichten**. Jede Schicht hat:

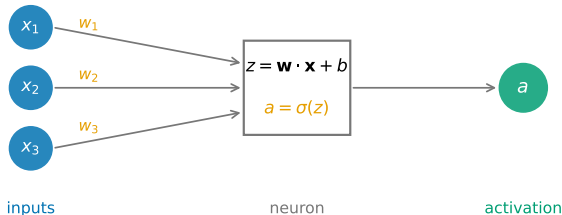
- **Eingaben**: Merkmalsvektor $\mathbf{x}$ (oder Ausgabe der vorigen Schicht)
- **Gewichtsmatrix $\mathbf{W}$** : gelernte Parameter, Eingaben $\rightarrow$ Neuronen
- **Bias-Vektor $\mathbf{b}$** : gelernte Verschiebung
- **Gewichtete Summe**: $\mathbf{z} = \mathbf{W}^T \mathbf{x} + \mathbf{b}$
- **Aktivierungsfunktion $\sigma(\cdot)$** : nicht-lineare Transformation auf $\mathbf{z}$

Neural Network Architektur: Schichten und Gewichte

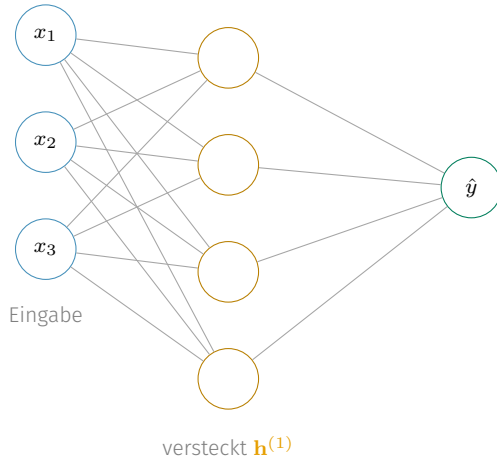
Ein neuronales Netz ist ein Stapel von **Schichten**. Jede Schicht hat:

- **Eingaben**: Merkmalsvektor $\mathbf{x}$ (oder Ausgabe der vorigen Schicht)
- **Gewichtsmatrix** $\mathbf{W}$: gelernte Parameter, Eingaben $\rightarrow$ Neuronen
- **Bias-Vektor** $\mathbf{b}$: gelernte Verschiebung
- **Gewichtete Summe**: $z = \mathbf{W}^T \mathbf{x} + \mathbf{b}$
- **Aktivierungsfunktion** $\sigma(\cdot)$: nicht-lineare Transformation auf z

Beispiel: ein Neuron mit 3 Eingaben:

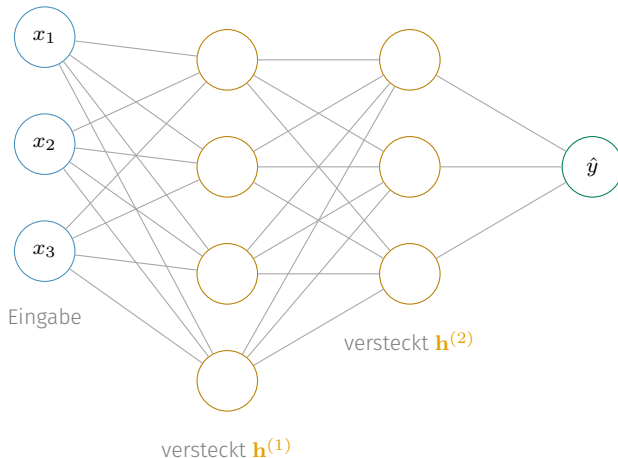


Vom Neuron zum Netz



1 Schicht: $\mathbf{h} = \sigma(W^T \mathbf{x} + \mathbf{b})$. Jede Zeile von W ist der Gewichtsvektor eines Neurons.

Vom Neuron zum Netz



Tief: $\mathbf{h}^{(1)} \rightarrow \mathbf{h}^{(2)} \rightarrow \hat{y}$. Jede Schicht speist die nächste; Tiefe erlaubt es, komplexe Merkmale Schritt für Schritt aufzubauen.

Warum brauchen wir Aktivierungsfunktionen?

Stapelt man nur lineare Transformationen (gewichtete Summen), bleibt das gesamte Netz **linear**!

Warum brauchen wir Aktivierungsfunktionen?

Stapelt man nur lineare Transformationen (gewichtete Summen), bleibt das gesamte Netz **linear**!

Mathematische Tatsache: Die Komposition linearer Funktionen ist linear

$$\text{Schicht 2}(\text{Schicht 1}(\mathbf{x})) = \mathbf{W}_2^T (\mathbf{W}_1^T \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\text{immer noch linear in } \mathbf{x})$$

$$\begin{pmatrix} w_1 & w_2 \\ w_3 & w_4 \end{pmatrix} \cdot \left(\begin{pmatrix} w_1 & w_2 \\ w_3 & w_4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right) + \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}$$

Warum brauchen wir Aktivierungsfunktionen?

Stapelt man nur lineare Transformationen (gewichtete Summen), bleibt das gesamte Netz **linear**!

Mathematische Tatsache: Die Komposition linearer Funktionen ist linear

$$\text{Schicht 2}(\text{Schicht 1}(\mathbf{x})) = \mathbf{W}_2^T(\mathbf{W}_1^T \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\text{immer noch linear in } \mathbf{x})$$

Lösung: Nach jeder Schicht eine **nicht-lineare Aktivierungsfunktion** $\sigma(\cdot)$ einfügen:

$$\mathbf{h} = \sigma(\mathbf{W}^T \mathbf{x} + \mathbf{b})$$

Nun ist: $\sigma(\text{Schicht 1}(\text{Schicht 2}(\mathbf{x})))$ **nicht-linear**!

Warum brauchen wir Aktivierungsfunktionen?

Stapelt man nur lineare Transformationen (gewichtete Summen), bleibt das gesamte Netz **linear**!

Mathematische Tatsache: Die Komposition linearer Funktionen ist linear

$$\text{Schicht 2}(\text{Schicht 1}(\mathbf{x})) = \mathbf{W}_2^T(\mathbf{W}_1^T \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\text{immer noch linear in } \mathbf{x})$$

Lösung: Nach jeder Schicht eine **nicht-lineare Aktivierungsfunktion** $\sigma(\cdot)$ einfügen:

$$\mathbf{h} = \sigma(\mathbf{W}^T \mathbf{x} + \mathbf{b})$$

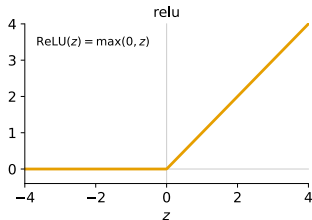
Nun ist: $\sigma(\text{Schicht 1}(\text{Schicht 2}(\mathbf{x})))$ **nicht-linear**!

Mächtigkeit: Mit genügend Schichten und Aktivierungsfunktionen lässt sich **jede** nicht-lineare Funktion lernen!

Häufige Aktivierungsfunktionen

ReLU:

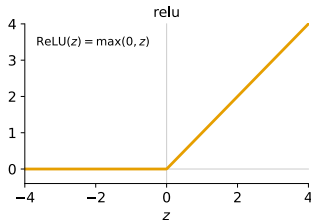
- einfach und schnell
- Standardwahl für versteckte Schichten
- Problem: „tote Neuronen“



Häufige Aktivierungsfunktionen

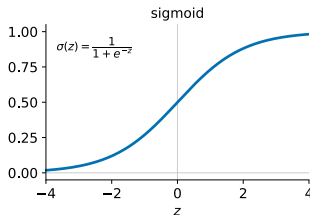
ReLU:

- einfach und schnell
- Standardwahl für versteckte Schichten
- Problem: „tote Neuronen“



Sigmoid:

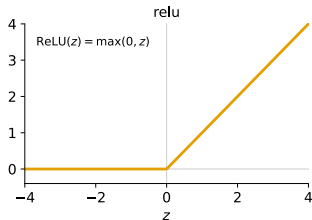
- Ausgaben zwischen 0 und 1
- gut für Ausgabeschichten
- langsames Training



Häufige Aktivierungsfunktionen

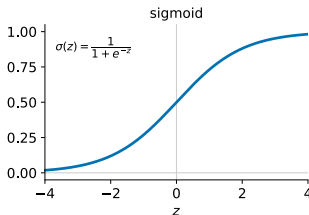
ReLU:

- einfach und schnell
- Standardwahl für versteckte Schichten
- Problem: „tote Neuronen“



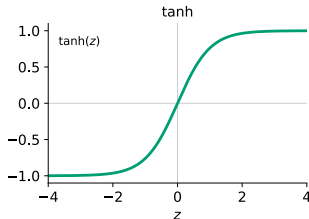
Sigmoid:

- Ausgaben zwischen 0 und 1
- gut für Ausgabeschichten
- langsames Training



Tanh: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

- Ausgaben zwischen -1 und 1
- stärkere Update-Signale
- gut für versteckte Schichten



Zusammenfassung

Ein neuronales Netz stapelt viele
Linear-und-Aktivierungs-Einheiten.

Jedes *Neuron*: $a = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$.

Die **Nichtlinearität** σ ist es, die **Tiefe** sinnvoll macht.

Mehr Schichten = mehr Kapazität = höheres Overfitting-Risiko
(wie wir in der Trainings-Lektion sehen werden).

Zusammenfassung

Ein neuronales Netz stapelt viele
Linear-und-Aktivierungs-Einheiten.

Jedes *Neuron*: $a = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$.

Die **Nichtlinearität** σ ist es, die **Tiefe** sinnvoll macht.

Mehr Schichten = mehr Kapazität = höheres Overfitting-Risiko
(wie wir in der Trainings-Lektion sehen werden).

- Logistische Regression = kleinstes Neuronales Netz mit Sigmoid-Ausgabe.
- Lineare Regression = kleinstes Neuronales Netz ohne Ausgabe-Aktivierung.



2. Theoretische Übungen

Übung 1: Der Kollaps

Eine Studentin zeigt ein „tiefes“ Netz: 5 gestapelte lineare Abbildungen $\mathbf{h}^{(\ell)} = W_{\ell}^T \mathbf{h}^{(\ell-1)} + \mathbf{b}_{\ell}$, mit $\mathbf{h}^{(0)} = \mathbf{x}$. Aktivierungsfunktionen wurden vergessen.

- (a) Man schreibe die Ausgabe des Netzes als eine einzige Formel in $\mathbf{x}$.
- (b) Man zeige, dass das Netz nicht ausdrucksstärker ist als eine *einzelne* lineare Einheit. Wie lauten die äquivalente Gewichtsmatrix und der äquivalente Bias?

Übung 1: Der Kollaps

Eine Studentin zeigt ein „tiefes“ Netz: 5 gestapelte lineare Abbildungen $\mathbf{h}^{(\ell)} = W_{\ell}^T \mathbf{h}^{(\ell-1)} + \mathbf{b}_{\ell}$, mit $\mathbf{h}^{(0)} = \mathbf{x}$. Aktivierungsfunktionen wurden vergessen.

- (a) Man schreibe die Ausgabe des Netzes als eine einzige Formel in $\mathbf{x}$.
- (b) Man zeige, dass das Netz nicht ausdrucksstärker ist als eine *einzelne* lineare Einheit. Wie lauten die äquivalente Gewichtsmatrix und der äquivalente Bias?

❗ (a) Schichtweises Einsetzen ergibt:
$$\hat{y} = W_5^T (W_4^T (W_3^T (W_2^T (W_1^T \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_3) + \mathbf{b}_4) + \mathbf{b}_5.$$

(b)

Übung 1: Der Kollaps

Eine Studentin zeigt ein „tiefes“ Netz: 5 gestapelte lineare Abbildungen $\mathbf{h}^{(\ell)} = W_\ell^T \mathbf{h}^{(\ell-1)} + \mathbf{b}_\ell$, mit $\mathbf{h}^{(0)} = \mathbf{x}$. Aktivierungsfunktionen wurden vergessen.

- (a) Man schreibe die Ausgabe des Netzes als eine einzige Formel in $\mathbf{x}$.
- (b) Man zeige, dass das Netz nicht ausdrucksstärker ist als eine *einzelne* lineare Einheit. Wie lauten die äquivalente Gewichtsmatrix und der äquivalente Bias?

❗ (a) Schichtweises Einsetzen ergibt:
 $\hat{y} = W_5^T (W_4^T (W_3^T (W_2^T (W_1^T \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_3) + \mathbf{b}_4) + \mathbf{b}_5.$

▽ 6 (b) Ausmultiplizieren liefert $\hat{y} = W' \mathbf{x} + \mathbf{b}'$ mit $W' = W_5^T W_4^T W_3^T W_2^T W_1^T$ und $\mathbf{b}' = W_5^T W_4^T W_3^T W_2^T \mathbf{b}_1 + \dots + W_5^T \mathbf{b}_4 + \mathbf{b}_5$. Die Komposition linearer Abbildungen ist linear, also berechnet eine *einzelne* Schicht mit $(W', \mathbf{b}')$ dieselbe Funktion.

Übung 2: Forward Pass berechnen

Gegeben ein zweischichtiges Netz mit:

- Eingabe: $\mathbf{x} = [1, 2]^T$
- Gewichte Schicht 1: $\mathbf{W}_1 = \begin{bmatrix} 0.5 & 0.2 \\ 0.3 & 0.1 \end{bmatrix}$, Bias: $\mathbf{b}_1 = [0.1, 0]^T$
- Aktivierung Schicht 1: ReLU
- Gewichte Schicht 2: $\mathbf{W}_2 = \begin{bmatrix} 0.4 & 0.6 \end{bmatrix}$, Bias: $b_2 = 0.2$
- Aktivierung Schicht 2: keine (lineare Ausgabe)

Übung 2: Forward Pass berechnen

Gegeben ein zweischichtiges Netz mit:

- Eingabe: $\mathbf{x} = [1, 2]^T$
- Gewichte Schicht 1: $\mathbf{W}_1 = \begin{bmatrix} 0.5 & 0.2 \\ 0.3 & 0.1 \end{bmatrix}$, Bias: $\mathbf{b}_1 = [0.1, 0]^T$
- Aktivierung Schicht 1: ReLU
- Gewichte Schicht 2: $\mathbf{W}_2 = \begin{bmatrix} 0.4 & 0.6 \end{bmatrix}$, Bias: $b_2 = 0.2$
- Aktivierung Schicht 2: keine (lineare Ausgabe)

Man berechne den Vorwärtsthroughlauf: $\mathbf{z}_1 \rightarrow \mathbf{h}_1 \rightarrow z_2 \rightarrow \hat{y}$

Übung 2: Forward Pass berechnen

Gegeben ein zweischichtiges Netz mit:

- Eingabe: $\mathbf{x} = [1, 2]^T$
- Gewichte Schicht 1: $\mathbf{W}_1 = \begin{bmatrix} 0.5 & 0.2 \\ 0.3 & 0.1 \end{bmatrix}$, Bias: $\mathbf{b}_1 = [0.1, 0]^T$
- Aktivierung Schicht 1: ReLU
- Gewichte Schicht 2: $\mathbf{W}_2 = \begin{bmatrix} 0.4 & 0.6 \end{bmatrix}$, Bias: $b_2 = 0.2$
- Aktivierung Schicht 2: keine (lineare Ausgabe)

Man berechne den Vorwärtsthroughlauf: $\mathbf{z}_1 \rightarrow \mathbf{h}_1 \rightarrow z_2 \rightarrow \hat{y}$

❗ $\mathbf{z}_1 = \mathbf{W}_1^T \mathbf{x} + \mathbf{b}_1 = \begin{bmatrix} 0.5 & 0.3 \\ 0.2 & 0.1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1.2 \\ 0.4 \end{bmatrix}$

$\mathbf{h}_1 = \text{ReLU}(\mathbf{z}_1) = \begin{bmatrix} 1.2 \\ 0.4 \end{bmatrix}$ (beide positiv, also unverändert)

$z_2 = \mathbf{W}_2^T \mathbf{h}_1 + b_2 = [0.4, 0.6] \begin{bmatrix} 1.2 \\ 0.4 \end{bmatrix} + 0.2 = 0.48 + 0.24 + 0.2 = 0.92$

$$w_1 \cdot x + b_1 = \begin{pmatrix} 1 \\ 0.5 \end{pmatrix}$$

$$\text{ReLU} \left(\begin{pmatrix} 1 \\ 0.5 \end{pmatrix} \right) = \begin{pmatrix} 1 \\ 0.5 \end{pmatrix}$$

$$w_2 \cdot a_1 = 0.4 + 0.3 = 0.7$$

$$0.7 + b = \underline{\underline{0.9}}$$

3. Programmierübung

Programmierübung

Sie haben nun die Kenntnisse, die folgenden Übungen zu lösen:

<https://expert.ethz.ch/enrolled/SS26/mavt2/exercises>

■ Spirals

4. Training

Was ist Training?

Ein Modell zu **trainieren** bedeutet: jene Gewichte $\mathbf{w}$ wählen, die einen Verlust $\mathcal{L}(\mathbf{w})$ auf den *Trainingsdaten minimieren*.

Die Matrizen

Was ist Training?

Ein Modell zu **trainieren** bedeutet: jene Gewichte $\mathbf{w}$ wählen, die einen Verlust $\mathcal{L}(\mathbf{w})$ auf den *Trainingsdaten minimieren*.

Doch ein Modell, das auf den Trainingsdaten gut abschneidet, kann trotzdem unbrauchbar sein.

- Entscheidend ist, wie es sich auf Daten verhält, die es *nicht* gesehen hat.

Was ist Training?

Ein Modell zu **trainieren** bedeutet: jene Gewichte $\mathbf{w}$ wählen, die einen Verlust $\mathcal{L}(\mathbf{w})$ auf den *Trainingsdaten minimieren*.

Doch ein Modell, das auf den Trainingsdaten gut abschneidet, kann trotzdem unbrauchbar sein.

- Entscheidend ist, wie es sich auf Daten verhält, die es *nicht* gesehen hat.

Geringer Trainingsverlust $\neq$ gutes Modell.

Trainings- und Validierungsdaten

Man teilt die Daten in zwei Teile auf.

Trainings- und Validierungsdaten

Man teilt die Daten in zwei Teile auf.

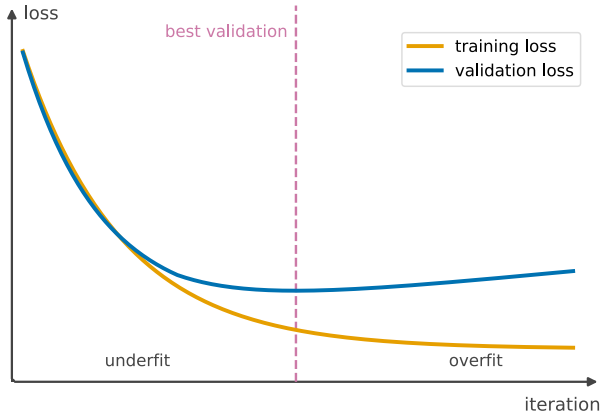
- **Trainingsmenge:** das Modell *sieht* diese Daten und passt seine Gewichte daran an.

Trainings- und Validierungsdaten

Man teilt die Daten in zwei Teile auf.

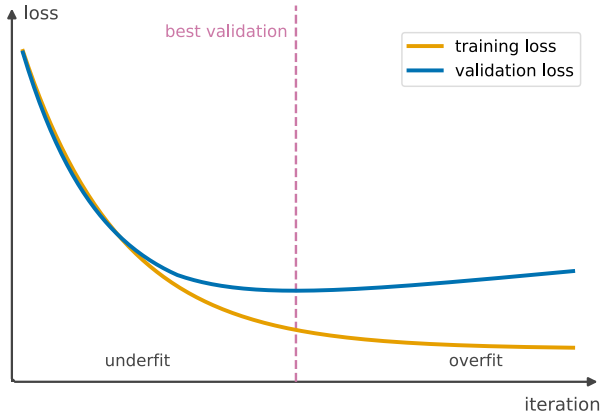
- **Trainingsmenge**: das Modell *sieht* diese Daten und passt seine Gewichte daran an.
- **Validierungsmenge**: das Modell *sieht* diese Daten während des Trainings *nie*. Sie dient dazu, zu prüfen, wie gut das Modell generalisiert. 

Verlustkurven



Nach jedem Trainingsschritt wird der Verlust auf *beiden* Mengen ausgewertet.

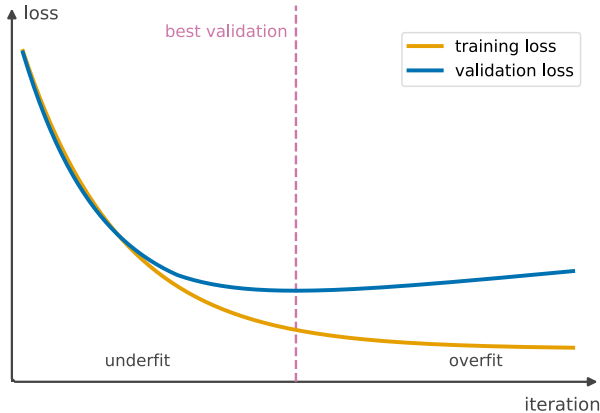
Verlustkurven



Nach jedem Trainingsschritt wird der Verlust auf *beiden* Mengen ausgewertet.

- Anfangs: beide fallen gemeinsam.

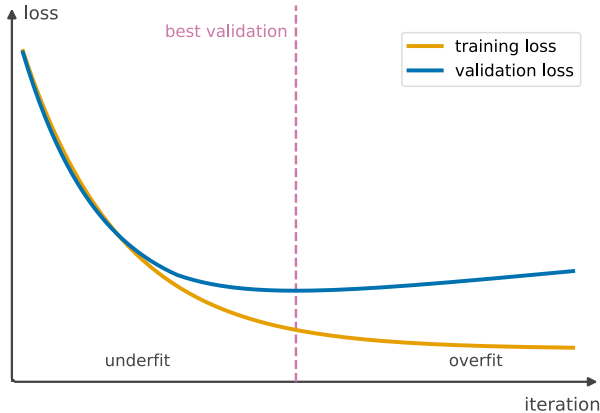
Verlustkurven



Nach jedem Trainingsschritt wird der Verlust auf *beiden* Mengen ausgewertet.

- Anfangs: beide fallen gemeinsam.
- Später: **Trainingsverlust** fällt weiter, **Validierungsverlust** fällt nicht mehr und *steigt*.

Verlustkurven



Nach jedem Trainingsschritt wird der Verlust auf *beiden* Mengen ausgewertet.

- Anfangs: beide fallen gemeinsam.
- Später: **Trainingsverlust** fällt weiter, **Validierungsverlust** fällt nicht mehr und *steigt*.
- Die **Lücke** ist das Signal.

Unteranpassung, gut, Überanpassung



Gleiche Daten, drei Modellkomplexitäten.

Links: zu einfach, die Gerade folgt dem Trend nicht (*Unteranpassung*: hoher Verlust überall).

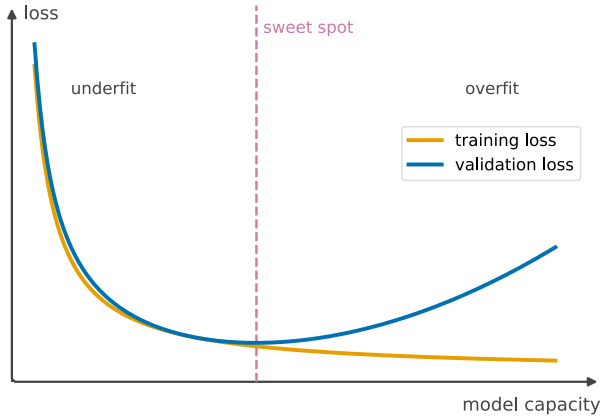
Mitte: passend, erfasst das Muster und ignoriert das Rauschen.

Rechts: zu flexibel, jagt jedem Punkt nach (*Überanpassung*: tiefer Trainings, hoher Validierungsverlust).

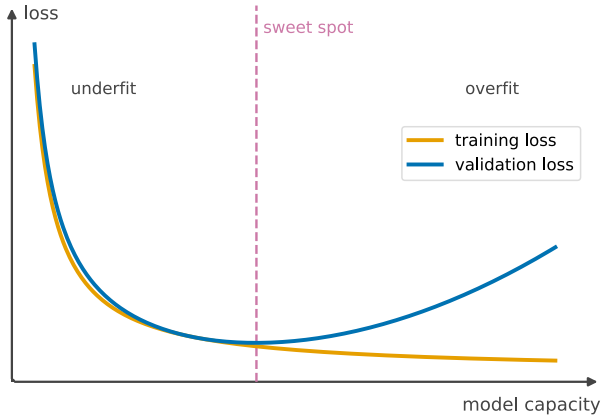
"interpretiert ein Muster, das
es gar nicht gibt."

Modellkapazität

Kapazität = wie flexibel ein Modell ist.



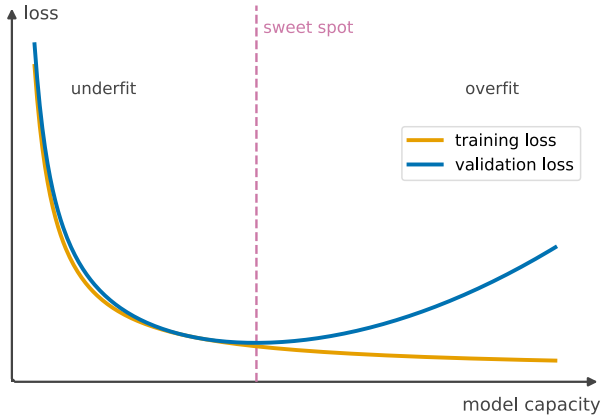
Modellkapazität



Kapazität = wie flexibel ein Modell ist.

- Geringe Kapazität $\Rightarrow$ Unteranpassung.

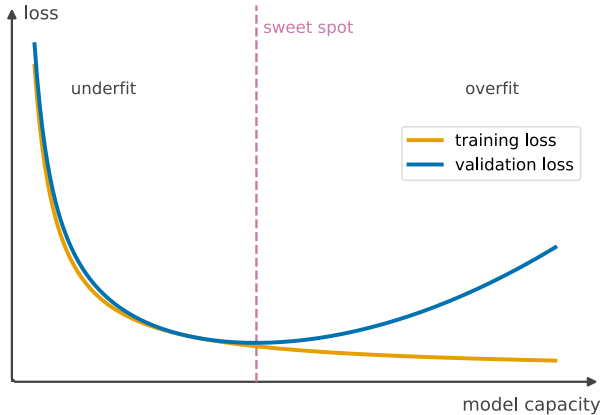
Modellkapazität



Kapazität = wie flexibel ein Modell ist.

- Geringe Kapazität $\Rightarrow$ Unteranpassung.
- Hohe Kapazität $\Rightarrow$ Überanpassung.

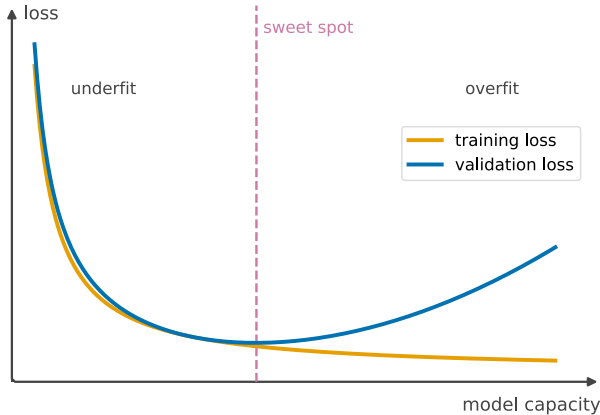
Modellkapazität



Kapazität = wie flexibel ein Modell ist.

- Geringe Kapazität $\Rightarrow$ Unteranpassung.
- Hohe Kapazität $\Rightarrow$ Überanpassung.
- Optimum dazwischen.

Modellkapazität



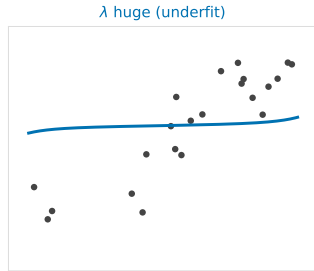
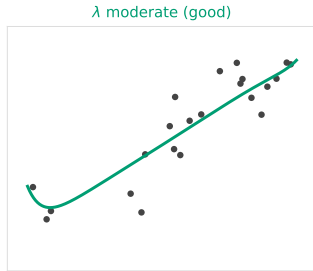
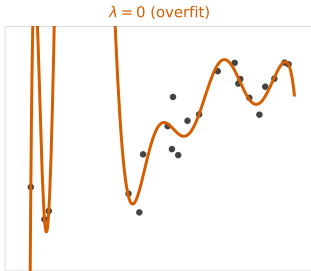
Kapazität = wie flexibel ein Modell ist.

- Geringe Kapazität $\Rightarrow$ Unteranpassung.
- Hohe Kapazität $\Rightarrow$ Überanpassung.
- Optimum dazwischen.

Stellschrauben für die Kapazität: *Grad* eines Polynoms, k bei k -NN, *Tiefe* eines Baumes (wie wir noch sehen werden), *Schichten* eines Netzes.

"Architektur"
 $\rightarrow$ Hyperparameter.

Regularisierung: ein Bild



Gleiche Daten, gleiche Modellfamilie (Polynom vom Grad 12), drei Werte der **Regularisierungsstärke** λ . Der Hebel λ bestimmt, wie stark grosse Gewichte bestraft werden.

Regularisierung: die Formel

Zum Verlust wird ein Strafterm für grosse Gewichte addiert:

$$\mathcal{L}_{\text{reg}}(\mathbf{w}) = \underbrace{\mathcal{L}(\mathbf{w})}_{\text{Datenanpassung}} + \underbrace{\lambda \|\mathbf{w}\|^2}_{\text{Strafterm}}$$

- Grosse Gewichte $\Rightarrow$ hoher Strafterm $\Rightarrow$ der Optimierer bevorzugt *kleine* Gewichte.
- Kleine Gewichte $\Rightarrow$ weniger zappeliges Modell $\Rightarrow$ weniger Überanpassung.

Regularisierung: die Formel

Zum Verlust wird ein Strafterm für grosse Gewichte addiert:

$$\mathcal{L}_{\text{reg}}(\mathbf{w}) = \underbrace{\mathcal{L}(\mathbf{w})}_{\text{Datenanpassung}} + \underbrace{\lambda \|\mathbf{w}\|^2}_{\text{Strafterm}}$$

- Grosse Gewichte $\Rightarrow$ hoher Strafterm $\Rightarrow$ der Optimierer bevorzugt *kleine* Gewichte.
- Kleine Gewichte $\Rightarrow$ weniger zappeliges Modell $\Rightarrow$ weniger Überanpassung.

Das ist L_2 -Regularisierung (auch Ridge genannt). Die Variante L_1 verwendet $\lambda \sum_d |w_d|$ und drückt manche Gewichte exakt auf null.

Parameter vs. Hyperparameter

Parameter	Hyperparameter
die Gewichte $\mathbf{w}$ <i>gelernt</i> während des Trainings	z. B. λ , Polynomgrad, k , Lernrate <i>gewählt</i> <u>vor dem Training</u>
bestimmt durch Verlust- minimierung auf der Trainings- menge	bestimmt auf der Validierungs- menge

Parameter vs. Hyperparameter

Parameter	Hyperparameter
die Gewichte $\mathbf{w}$ <i>gelernt</i> während des Trainings	z. B. λ , Polynomgrad, k , Lernrate <i>gewählt</i> vor dem Training
bestimmt durch Verlust- minimierung auf der Trainings- menge	bestimmt auf der Validierungs- menge

Parameter sind, was das Modell lernt. Hyperparameter sind die Stellschrauben, die wir vorgeben.

Hyperparameter-Tuning

Das einfachste Rezept ist die **Gittersuche**:

Hyperparameter-Tuning

Das einfachste Rezept ist die **Gittersuche**:

1. Eine Liste von Kandidaten wählen, z. B. $\lambda \in \{0.001, 0.01, 0.1, 1, 10\}$.

Hyperparameter-Tuning

Das einfachste Rezept ist die **Gittersuche**:

1. Eine Liste von Kandidaten wählen, z. B. $\lambda \in \{0.001, 0.01, 0.1, 1, 10\}$.
2. Für jeden Kandidaten: auf der Trainingsmenge trainieren, auf der Validierungsmenge auswerten.

Hyperparameter-Tuning

Das einfachste Rezept ist die **Gittersuche**:

1. Eine Liste von Kandidaten wählen, z. B. $\lambda \in \{0.001, 0.01, 0.1, 1, 10\}$.
2. Für jeden Kandidaten: auf der Trainingsmenge trainieren, auf der Validierungsmenge auswerten.
3. Den Kandidaten mit dem geringsten Validierungsverlust behalten.

Hyperparameter-Tuning

Das einfachste Rezept ist die **Gittersuche**:

1. Eine Liste von Kandidaten wählen, z. B. $\lambda \in \{0.001, 0.01, 0.1, 1, 10\}$.
2. Für jeden Kandidaten: auf der Trainingsmenge trainieren, auf der Validierungsmenge auswerten.
3. Den Kandidaten mit dem geringsten Validierungsverlust behalten.
4. Den endgültigen Verlust auf der Testmenge berichten, und zwar ein einziges Mal.

Hyperparameter-Tuning

Das einfachste Rezept ist die **Gittersuche**:

1. Eine Liste von Kandidaten wählen, z. B. $\lambda \in \{0.001, 0.01, 0.1, 1, 10\}$.
2. Für jeden Kandidaten: auf der **Trainings**menge trainieren, auf der **Validierung**smenge auswerten.
3. Den Kandidaten mit dem geringsten **Validierung**sverlust behalten.
4. Den endgültigen Verlust auf der **Test**menge berichten, und zwar ein einziges Mal.

Ist die Validierungsmenge klein und verrauscht, kann der gewählte Hyperparameter unglücklich ausfallen. Die Kreuzvalidierung (wie zuvor gesehen) schafft Abhilfe.

Zusammenfassung

! Den Validierungsverlust im Auge behalten, nicht nur den Trainingsverlust.

Regularisierung und passende Kapazität bringen einen ins Optimum.

! Hyperparameter auf der Validierungs, nicht der Trainingsmenge bestimmen.

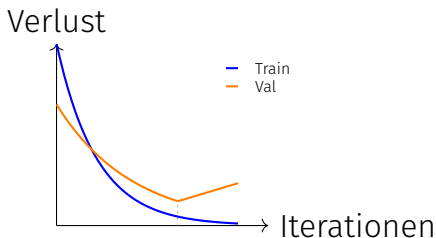
5. Theoretische Übungen

Übung 1: Verlustkurven lesen

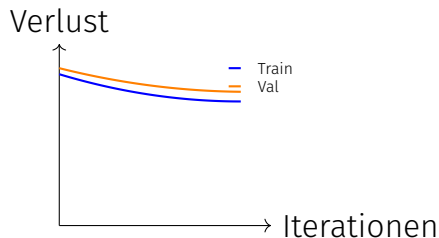
Für jedes Szenario sind Trainings- und Validierungsverlust gegen die Iterationszahl aufgetragen.

- (a) Trainingsverlust geht fast auf null; Validierungsverlust fällt zuerst, *steigt* dann und endet weit über dem Trainingsverlust.
- (b) Beide stagnieren früh auf einem hohen Wert.

Szenario (a):

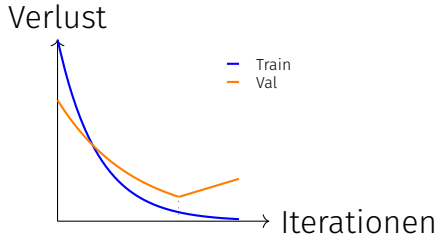


Szenario (b):

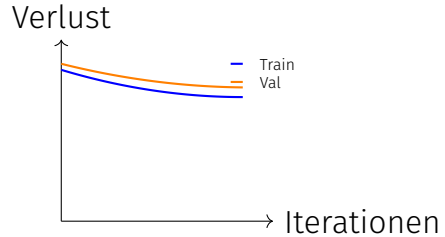


Übung 1: Verlustkurven lesen

Szenario (a):



Szenario (b):

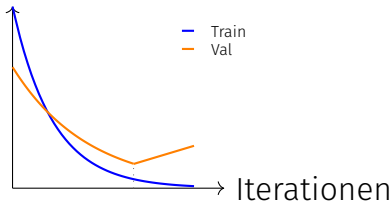


Für jedes: **Diagnose** und **empfohlene Massnahme**.

Übung 1: Verlustkurven lesen

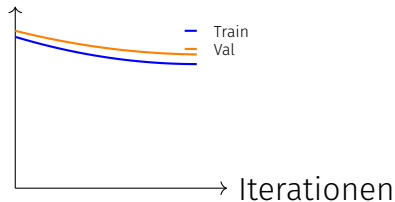
Szenario (a):

Verlust



Szenario (b):

Verlust



Für jedes: **Diagnose** und **empfohlene Massnahme**.

- ❗ (a) **Überanpassung**. Massnahmen: Early Stopping, λ erhöhen, Kapazität senken oder mehr Daten.
- (b) **Unteranpassung**. Massnahmen: flexibleres Modell, λ senken, mehr Epochen oder Merkmale.

Übung 2: Parameter oder Hyperparameter?

Man ordne jeden Punkt als **Parameter** (gelernt durch Verlustminimierung auf der Trainingsmenge) oder **Hyperparameter** (vor dem Training gewählt, auf der Validierungsmenge bestimmt) ein.

- (a) Das Gewicht w_3 in der linearen Regression.
- (b) Die Regularisierungsstärke λ .
- (c) Der Bias w_0 .
- (d) Die Lernrate des Optimierers.
- (e) Die Anzahl k der Nachbarn bei k -NN.
- (f) Die Anzahl Schichten eines neuronalen Netzes.

Übung 2: Parameter oder Hyperparameter?

Man ordne jeden Punkt als **Parameter** (gelernt durch Verlustminimierung auf der Trainingsmenge) oder **Hyperparameter** (vor dem Training gewählt, auf der Validierungsmenge bestimmt) ein.

- (a) Das Gewicht w_3 in der linearen Regression.
- (b) Die Regularisierungsstärke λ .
- (c) Der Bias w_0 .
- (d) Die Lernrate des Optimierers.
- (e) Die Anzahl k der Nachbarn bei k -NN.
- (f) Die Anzahl Schichten eines neuronalen Netzes.

❗ **Parameter:** (a) w_3 , (c) w_0 .

Hyperparameter: (b) λ , (d) Lernrate, (e) k , (f) Anzahl Schichten.

Die Unterscheidung gilt *allgemein*: jedes Modell hat beide Arten.

6. Programmierübung

Programmierung

Sie haben nun die Kenntnisse, die folgenden Übungen zu lösen:

<https://expert.ethz.ch/enrolled/SS26/mavt2/exercises>

- **Diabetes with regression trees and grid search** (*complete after the Decision Trees session*)
- **Sine Regression**

7. Entscheidungsbäume

Motivation: ein Flussdiagramm aus Ja/Nein-Fragen

Die Idee in einem Satz:

*Eine Folge von Ja/Nein-Fragen über die **Merkmale** stellen und den Antworten zur Vorhersage folgen.*

Motivation: ein Flussdiagramm aus Ja/Nein-Fragen

Die Idee in einem Satz:

*Eine Folge von Ja/Nein-Fragen über die **Merkmale** stellen und den Antworten zur Vorhersage folgen.*

Das ist das ganze Modell, ein **Flussdiagramm**, von oben nach unten gelesen.

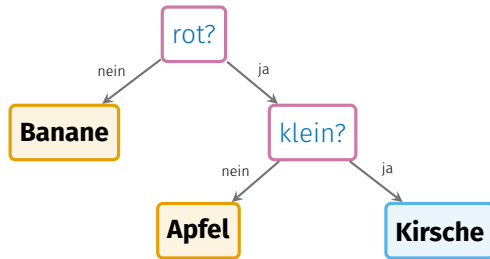
Motivation: ein Flussdiagramm aus Ja/Nein-Fragen

Die Idee in einem Satz:

*Eine Folge von Ja/Nein-Fragen über die **Merkmale** stellen und den Antworten zur Vorhersage folgen.*

Das ist das ganze Modell, ein **Flussdiagramm**, von oben nach unten gelesen.

- Keine Gleichungen, keine Parameter zum Anpassen.
- Das Modell ist das Flussdiagramm selbst.

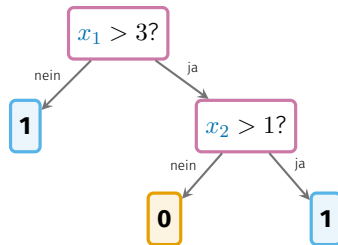


Zwei Fragen klassifizieren die Frucht.

Aufbau eines Entscheidungsbaums

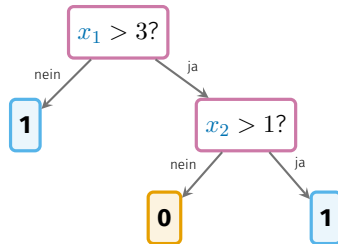
■ Innerer Knoten (violett):

Ja/Nein-Frage über *ein* Merkmal, z. B.
„ $x_j > t_j$ “. Der oberste ist die **Wurzel**.



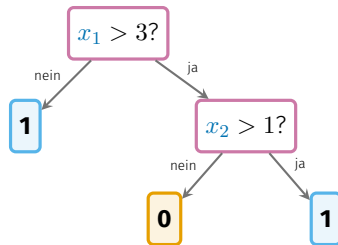
Aufbau eines Entscheidungsbaums

- **Innerer Knoten** (violett):
Ja/Nein-Frage über *ein* Merkmal, z. B.
„ $x_j > t_j$ “. Der oberste ist die **Wurzel**.
- **Blatt** (orange oder blau, eine Farbe pro Klasse): eine Vorhersage $\hat{y}$ am unteren Ende des Baums.



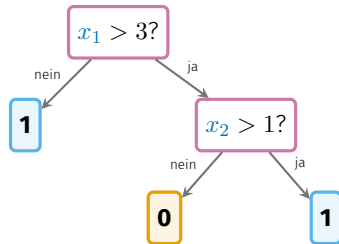
Aufbau eines Entscheidungsbaums

- **Innerer Knoten** (violett):
Ja/Nein-Frage über *ein* Merkmal, z. B. „ $x_j > t_j$ “. Der oberste ist die **Wurzel**.
- **Blatt** (orange oder blau, eine Farbe pro Klasse): eine Vorhersage $\hat{y}$ am unteren Ende des Baums.
- **Tiefe**: längster Pfad von der Wurzel zu einem Blatt, also die maximale Anzahl Fragen pro Datenpunkt.

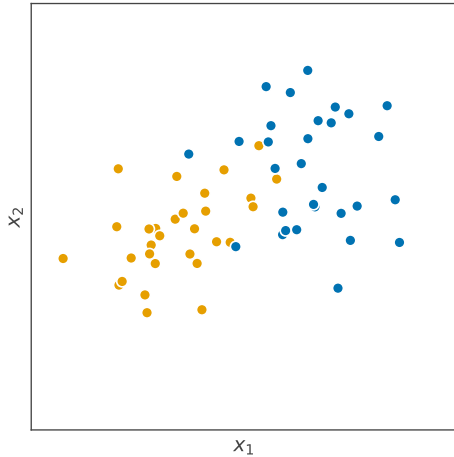


Aufbau eines Entscheidungsbaums

- **Innerer Knoten** (violett):
Ja/Nein-Frage über *ein* Merkmal, z. B. „ $x_j > t_j'$ “. Der oberste ist die **Wurzel**.
- **Blatt** (orange oder blau, eine Farbe pro Klasse): eine Vorhersage $\hat{y}$ am unteren Ende des Baums.
- **Tiefe**: längster Pfad von der Wurzel zu einem Blatt, also die maximale Anzahl Fragen pro Datenpunkt.
- **Vorhersage**: von der Wurzel hinab den Antworten folgen, bis ein Blatt erreicht ist. Den dort gespeicherten Wert ausgeben.

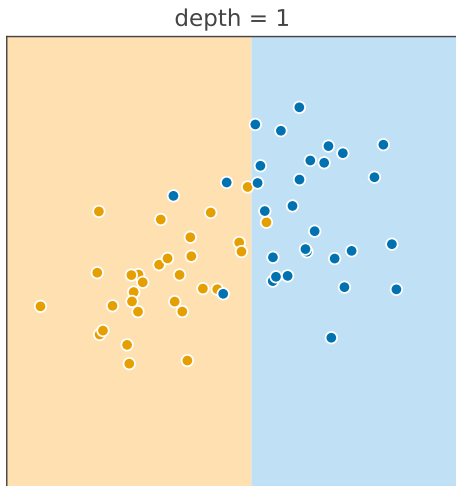


Entscheidungsregionen: achsenparallele Rechtecke



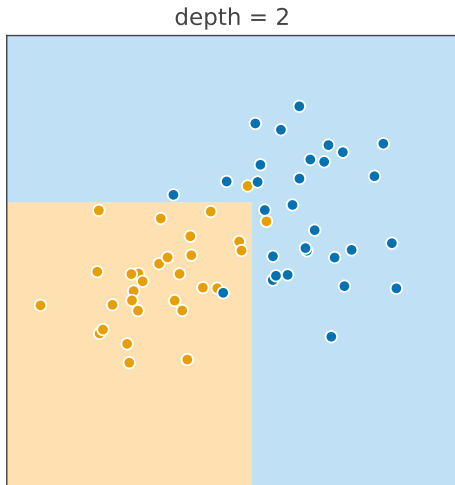
■ Zwei Klassen (A, B) in 2D.

Entscheidungsregionen: achsenparallele Rechtecke



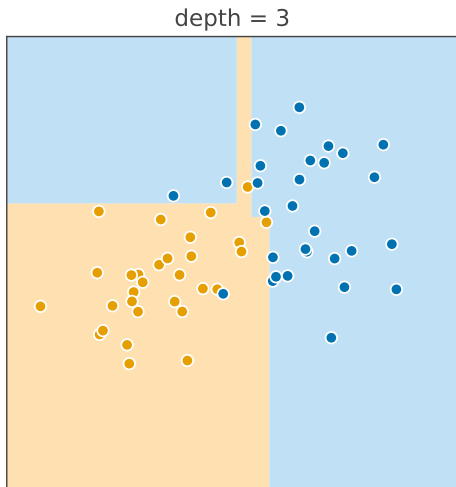
- Zwei Klassen (A, B) in 2D.
- **Tiefe 1:** ein Schnitt $\Rightarrow$ zwei Regionen.

Entscheidungsregionen: achsenparallele Rechtecke



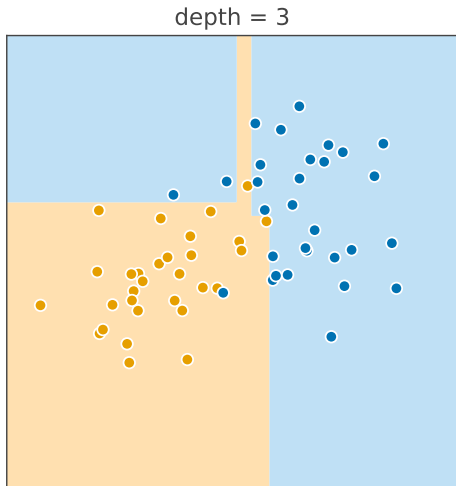
- Zwei Klassen (A, B) in 2D.
- **Tiefe 1:** ein Schnitt $\Rightarrow$ zwei Regionen.
- **Tiefe 2:** jede Region erneut teilbar $\Rightarrow$ bis zu vier.

Entscheidungsregionen: achsenparallele Rechtecke



- Zwei Klassen (A, B) in 2D.
- **Tiefe 1:** ein Schnitt $\Rightarrow$ zwei Regionen.
- **Tiefe 2:** jede Region erneut teilbar $\Rightarrow$ bis zu vier.
- **Tiefe 3:** bis zu acht Rechtecke.

Entscheidungsregionen: achsenparallele Rechtecke



- Zwei Klassen (A, B) in 2D.
- **Tiefe 1:** ein Schnitt $\Rightarrow$ zwei Regionen.
- **Tiefe 2:** jede Region erneut teilbar $\Rightarrow$ bis zu vier.
- **Tiefe 3:** bis zu acht Rechtecke.
- Zwei gleichwertige Sichten:
 - Flussdiagramm aus Ja/Nein-Fragen;
 - Zerlegung des **Merkmalsraums** (ein Rechteck pro Blatt).

Baum aufbauen: gierig, von oben

Der Baum wird *rekursiv* aufgebaut, beginnend bei der Wurzel mit **allen** Trainingsdaten:

Baum aufbauen: gierig, von oben

Der Baum wird *rekursiv* aufgebaut, beginnend bei der Wurzel mit **allen** Trainingsdaten:

1. Unter allen Kandidaten-Aufteilungen „ $x_j > t_j$ “ diejenige wählen, die die Klassen im aktuellen Knoten am besten trennt.

Baum aufbauen: gierig, von oben

Der Baum wird *rekursiv* aufgebaut, beginnend bei der Wurzel mit **allen** Trainingsdaten:

1. Unter allen Kandidaten-Aufteilungen „ $x_j > t_j$ “ diejenige wählen, die die Klassen im aktuellen Knoten am besten trennt.
2. Jeden Datenpunkt gemäss seiner Antwort an das linke oder rechte Kind weiterleiten.

Baum aufbauen: gierig, von oben

Der Baum wird *rekursiv* aufgebaut, beginnend bei der Wurzel mit **allen** Trainingsdaten:

1. Unter allen Kandidaten-Aufteilungen „ $x_j > t_j'$ “ diejenige wählen, die die **Klassen im aktuellen Knoten am besten trennt**.
2. Jeden Datenpunkt gemäss seiner Antwort an das **linke** oder **rechte** Kind weiterleiten.
3. In jedem Kind wiederholen, bis eine **Abbruchregel** erfüllt ist.

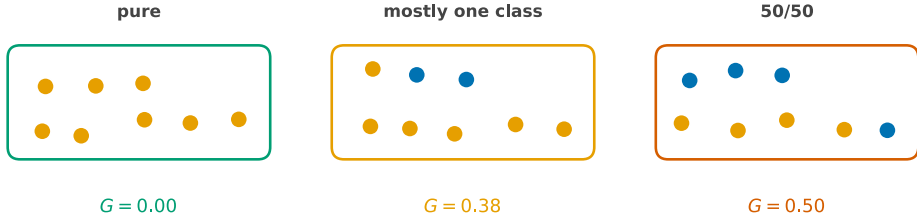
Baum aufbauen: gierig, von oben

Der Baum wird *rekursiv* aufgebaut, beginnend bei der Wurzel mit **allen** Trainingsdaten:

1. Unter allen Kandidaten-Aufteilungen „ $x_j > t_j$ “ diejenige wählen, die die **Klassen im aktuellen Knoten am besten trennt**.
2. Jeden Datenpunkt gemäss seiner Antwort an das **linke** oder **rechte** Kind weiterleiten.
3. In jedem Kind wiederholen, bis eine **Abbruchregel** erfüllt ist.

„Am besten“ muss präzisiert werden: man braucht ein numerisches **Aufteilungskriterium**, das misst, wie *gemischt* ein Knoten ist.

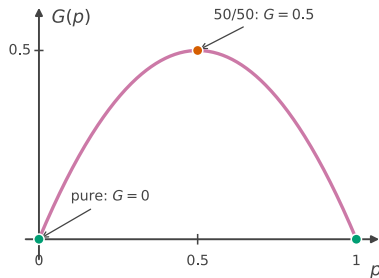
Unreinheit: “wie gemischt ist die Gruppe?”



Eine gute Aufteilung schickt (überwiegend) **eine** Klasse nach links und (überwiegend) **die andere** Klasse nach rechts, sodass die Kinder Richtung **reiner** Gruppen gedrängt werden.

Splitkriterium: der Gini-Index

- p_c = Anteil der Klasse c im Knoten.



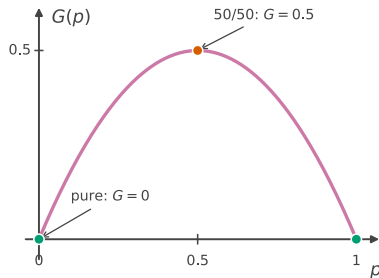
$$G = 2p(1-p)$$

p = Anteil positiver Klasse

Splitkriterium: der Gini-Index

- p_c = Anteil der Klasse c im Knoten.
- **Gini-Unreinheit** (K Klassen):

$$G = 1 - \sum_c p_c^2.$$



$$G = 2p(1-p)$$

p = Anteil positiver Klasse

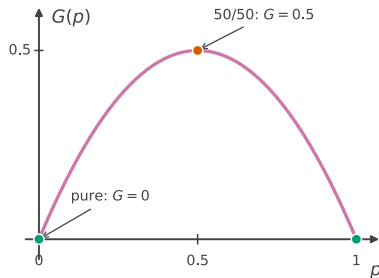
Splitkriterium: der Gini-Index

- p_c = Anteil der Klasse c im Knoten.

- **Gini-Unreinheit** (K Klassen):

$$G = 1 - \sum_c p_c^2.$$

- Zwei Klassen: $G = 2p(1-p)$.
Rein: $G = 0$. 50/50: $G = \frac{1}{2}$ (max.).



$$G = 2p(1-p)$$

p = Anteil positiver Klasse

Splitkriterium: der Gini-Index

- p_c = Anteil der Klasse c im Knoten.

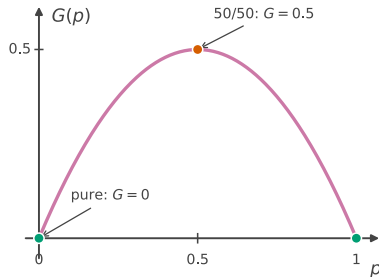
- **Gini-Unreinheit** (K Klassen):

$$G = 1 - \sum_c p_c^2.$$

- Zwei Klassen: $G = 2p(1-p)$.
Rein: $G = 0$. 50/50: $G = \frac{1}{2}$ (max.).
- **Split-Score** (gewichtet):

$$G_{\text{split}} = \frac{n_L}{n} G_L + \frac{n_R}{n} G_R.$$

Aufteilung mit **kleinstem** G_{split} wählen.



$$G = 2p(1-p)$$

p = Anteil positiver Klasse

Wann aufhören? Die Tiefe als zentraler Schalter

Abbruchregeln, also wann ein Knoten zum Blatt erklärt wird:

- eine gewählte **maximale Tiefe** ist erreicht;
- der Knoten enthält weniger als m Datenpunkte;
- der Knoten ist (fast) rein (G unter einem Schwellenwert).

Wann aufhören? Die Tiefe als zentraler Schalter

Abbruchregeln, also wann ein Knoten zum Blatt erklärt wird:

- eine gewählte **maximale Tiefe** ist erreicht;
- der Knoten enthält weniger als m Datenpunkte;
- der Knoten ist (fast) rein (G unter einem Schwellenwert).

Die **maximale Tiefe** ist der wichtigste Schalter (ein **Hyperparameter**, siehe Training-Lektion). Sie steuert, wie viele Fragen der Baum stellen darf, und damit die Komplexität der Entscheidungsgrenze.

Wann aufhören? Die Tiefe als zentraler Schalter

Abbruchregeln, also wann ein Knoten zum Blatt erklärt wird:

- eine gewählte **maximale Tiefe** ist erreicht;
- der Knoten enthält weniger als m Datenpunkte;
- der Knoten ist (fast) rein (G unter einem Schwellenwert).

Die **maximale Tiefe** ist der wichtigste Schalter (ein **Hyperparameter**, siehe Training-Lektion). Sie steuert, wie viele Fragen der Baum stellen darf, und damit die Komplexität der Entscheidungsgrenze.

- Flacher Baum $\Rightarrow$ zu wenige Fragen $\Rightarrow$ *Unteranpassung* (echte Struktur verfehlt).

Wann aufhören? Die Tiefe als zentraler Schalter

Abbruchregeln, also wann ein Knoten zum Blatt erklärt wird:

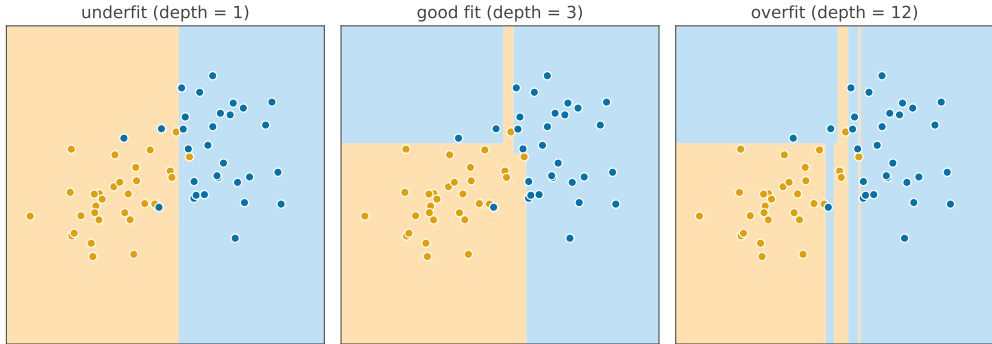
- eine gewählte **maximale Tiefe** ist erreicht;
- der Knoten enthält weniger als m Datenpunkte;
- der Knoten ist (fast) rein (G unter einem Schwellenwert).

Die **maximale Tiefe** ist der wichtigste Schalter (ein **Hyperparameter**, siehe Training-Lektion). Sie steuert, wie viele Fragen der Baum stellen darf, und damit die Komplexität der Entscheidungsgrenze.

- Flacher Baum $\Rightarrow$ zu wenige Fragen $\Rightarrow$ *Unteranpassung* (echte Struktur verfehlt).
- Sehr tiefer Baum $\Rightarrow$ jedes Blatt enthält einen einzelnen Trainingspunkt $\Rightarrow$ 100% auf Trainingsdaten, schlecht auf neuen Daten $\Rightarrow$ *Überanpassung* (Auswendiglernen).

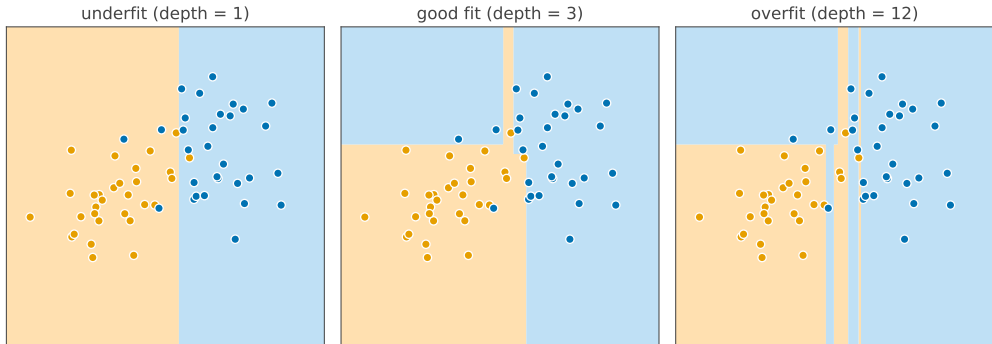
Aus der Training-Lektion: konkretes Beispiel für Über- und Unteranpassung.

Tiefe visuell: zu flach, gut, zu tief



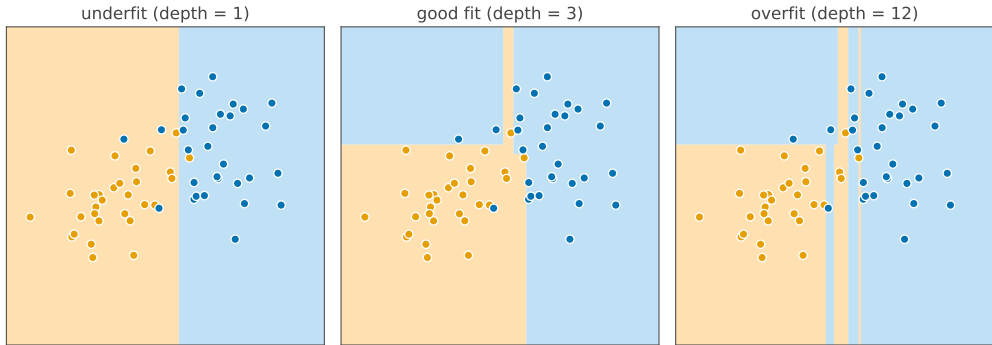
- Links: zu **flach**, um die Struktur zu erfassen.

Tiefe visuell: zu flach, gut, zu tief



- Links: zu **flach**, um die Struktur zu erfassen.
- Mitte: mittlere Tiefe, der **optimale Bereich**.

Tiefe visuell: zu flach, gut, zu tief



- Links: zu **flach**, um die Struktur zu erfassen.
- Mitte: mittlere Tiefe, der **optimale Bereich**.
- Rechts: tief, winzige Rechtecke um einzelne Punkte, also **Auswendiglernen**.

Massnahmen gegen Überanpassung

Drei Hyperparameter (wie in der Training-Lektion gesehen), welche die Kapazität des Baums begrenzen:

- **max_depth**: die Tiefe des Baums deckeln.

Massnahmen gegen Überanpassung

Drei Hyperparameter (wie in der Training-Lektion gesehen), welche die Kapazität des Baums begrenzen:

- **max_depth**: die Tiefe des Baums deckeln.
- **min_samples_leaf**: jedes Blatt muss mindestens m Trainingspunkte enthalten, sodass keine Blätter um einzelne Ausreisser entstehen.

Alle drei Schalter begrenzen, wie flexibel sich der Baum an die Trainingsdaten anpassen kann: ein weniger flexibler Baum verfehlt unter Umständen echte Struktur, ein sehr flexibler Baum lernt die Trainingspunkte auswendig und sagt auf neuen Daten schlecht voraus.

Massnahmen gegen Überanpassung

Drei Hyperparameter (wie in der Training-Lektion gesehen), welche die Kapazität des Baums begrenzen:

- **max_depth**: die Tiefe des Baums deckeln.
- **min_samples_leaf**: jedes Blatt muss mindestens m Trainingspunkte enthalten, sodass keine Blätter um einzelne Ausreisser entstehen.
- **Pruning**: einen tiefen Baum wachsen lassen und anschliessend Zweige *beschneiden*, die auf einer zurückgehaltenen Menge nichts bringen. (Nur als Idee; in dieser Vorlesung nicht implementiert.)

Alle drei Schalter begrenzen, wie flexibel sich der Baum an die Trainingsdaten anpassen kann: ein weniger flexibler Baum verfehlt unter Umständen echte Struktur, ein sehr flexibler Baum lernt die Trainingspunkte auswendig und sagt auf neuen Daten schlecht voraus.

Kategorische Merkmale

Echte Datensätze enthalten oft nicht-numerische Merkmale: *Land, Farbe, Blutgruppe*.

Kategorische Merkmale

Echte Datensätze enthalten oft nicht-numerische Merkmale: *Land, Farbe, Blutgruppe*.

Ein Baum verarbeitet sie *direkt*, denn eine Frage kann einfach nach der Kategoriezugehörigkeit fragen:

- „Land = CH?“
- „Farbe $\in$ {rot, orange}?“

Kategorische Merkmale

Echte Datensätze enthalten oft nicht-numerische Merkmale: *Land, Farbe, Blutgruppe*.

Ein Baum verarbeitet sie *direkt*, denn eine Frage kann einfach nach der Kategoriezugehörigkeit fragen:

- „Land = CH?“
- „Farbe $\in$ {rot, orange}?“
- Keine Vorverarbeitung des Merkmals nötig.
- Die Aufteilung bleibt für Menschen lesbar, eine echte praktische Stärke von Bäumen bei tabellarischen Daten mit gemischten Merkmalstypen.

Zusammenfassung

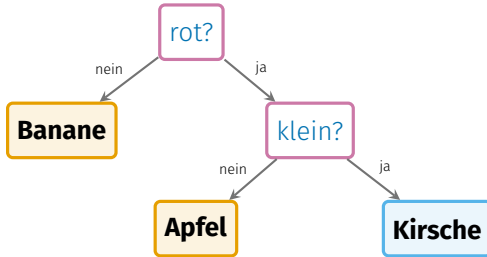
Ein Entscheidungsbaum ist ein Flussdiagramm
aus Ja/Nein-Fragen,
das den Merkmalsraum in achsenparallele
Rechtecke zerlegt.

Gierig wachsen lassen, Unreinheit minimieren;
Tiefe deckeln gegen Auswendiglernen.

8. Theoretische Übungen

Übung 1: Einen Baum durchlaufen

Mit dem Baum unten jede Frucht klassifizieren. Die Merkmale sind *rot?* und *klein?*, beide ja/nein.



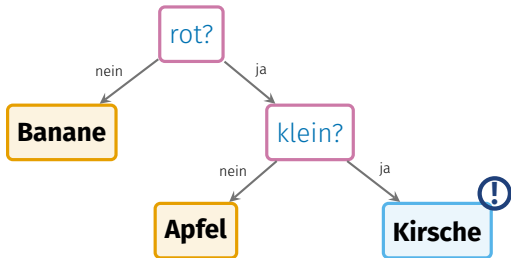
(a) $\mathbf{x}_1 = (\text{rot} = \text{nein}, \text{klein} = \text{ja})$

(b) $\mathbf{x}_2 = (\text{rot} = \text{ja}, \text{klein} = \text{nein})$

(c) $\mathbf{x}_3 = (\text{rot} = \text{ja}, \text{klein} = \text{ja})$

Übung 1: Einen Baum durchlaufen

Mit dem Baum unten jede Frucht klassifizieren. Die Merkmale sind *rot?* und *klein?*, beide ja/nein.



(a) $\mathbf{x}_1 = (\text{rot} = \text{nein}, \text{klein} = \text{ja})$

(b) $\mathbf{x}_2 = (\text{rot} = \text{ja}, \text{klein} = \text{nein})$

(c) $\mathbf{x}_3 = (\text{rot} = \text{ja}, \text{klein} = \text{ja})$



(a) rot? **nein** → **Banane**.

(b) rot? **ja** → klein? **nein** → **Apfel**.

(c) rot? **ja** → klein? **ja** → **Kirsche**.

Übung 2: Einen Gini-Split berechnen

Ein Knoten enthält 6 Datenpunkte: 4 **positive**, 2 **negative**. Eine Kandidaten-Aufteilung $x_1 > 3$ ergibt:

- linkes Kind ($n_L = 3$): 3 **pos**, 0 **neg**
- rechtes Kind ($n_R = 3$): 1 **pos**, 2 **neg**

Erinnerung: $G = 1 - p_+^2 - p_-^2$, mit $p_+ = \frac{\#_{\text{pos}}}{\#_{\text{gesamt}}}$ und $p_- = 1 - p_+$.

Übung 2: Einen Gini-Split berechnen

Ein Knoten enthält 6 Datenpunkte: 4 **positive**, 2 **negative**. Eine Kandidaten-Aufteilung $x_1 > 3$ ergibt:

- linkes Kind ($n_L = 3$): 3 **pos**, 0 **neg**
- rechtes Kind ($n_R = 3$): 1 **pos**, 2 **neg**

Erinnerung: $G = 1 - p_+^2 - p_-^2$, mit $p_+ = \frac{\#_{\text{pos}}}{\#_{\text{gesamt}}}$ und $p_- = 1 - p_+$.

- p_+ , p_- für den **Elternknoten** berechnen und daraus die Gini-Unreinheit G bestimmen.
- G_L und G_R für jedes Kind berechnen. (Was ist G , wenn $p_+ = 1$?)
- $G_{\text{split}} = \frac{n_L}{n} G_L + \frac{n_R}{n} G_R$ berechnen. Ist das eine gute Aufteilung?

Übung 2: Einen Gini-Split berechnen (Lösung)

❗ (a) Elternknoten: $p_+ = 4/6$, $p_- = 2/6$, also

$$G = 1 - (4/6)^2 - (2/6)^2 = 1 - \frac{16}{36} - \frac{4}{36} = \frac{16}{36} \approx 0.44.$$

Übung 2: Einen Gini-Split berechnen (Lösung)

❗ (a) Elternknoten: $p_+ = 4/6$, $p_- = 2/6$, also

$$G = 1 - (4/6)^2 - (2/6)^2 = 1 - \frac{16}{36} - \frac{4}{36} = \frac{16}{36} \approx 0.44.$$

(b) Links ist rein: $G_L = 0$. Rechts: $p_+ = 1/3$, $p_- = 2/3$, also

$$G_R = 1 - (1/3)^2 - (2/3)^2 = 1 - \frac{1}{9} - \frac{4}{9} = \frac{4}{9} \approx 0.44.$$

Übung 2: Einen Gini-Split berechnen (Lösung)

❗ (a) Elternknoten: $p_+ = 4/6$, $p_- = 2/6$, also

$$G = 1 - (4/6)^2 - (2/6)^2 = 1 - \frac{16}{36} - \frac{4}{36} = \frac{16}{36} \approx 0.44.$$

(b) Links ist rein: $G_L = 0$. Rechts: $p_+ = 1/3$, $p_- = 2/3$, also

$$G_R = 1 - (1/3)^2 - (2/3)^2 = 1 - \frac{1}{9} - \frac{4}{9} = \frac{4}{9} \approx 0.44.$$

(c) Gewichtete Kind-Unreinheit:

$$G_{\text{split}} = \frac{3}{6} \cdot 0 + \frac{3}{6} \cdot \frac{4}{9} = \frac{2}{9} \approx 0.22.$$

0.44 $\rightarrow$ 0.22 ist ein deutlicher Rückgang. Ja, das ist eine gute Aufteilung.

Übung 3: Tiefe diagnostizieren

Ein Student trainiert einen Entscheidungsbaum der Tiefe 20 auf $n = 100$ Punkten und berichtet

- Trainingsgenauigkeit = 100%,
- Validierungsgenauigkeit = 60%.

Übung 3: Tiefe diagnostizieren

Ein Student trainiert einen Entscheidungsbaum der Tiefe 20 auf $n = 100$ Punkten und berichtet

- Trainingsgenauigkeit = 100%,
 - Validierungsgenauigkeit = 60%.
- (a) Was ist die wahrscheinlichste Diagnose?
- (b) Was sollte als Nächstes probiert werden?

Übung 3: Tiefe diagnostizieren

Ein Student trainiert einen Entscheidungsbaum der Tiefe 20 auf $n = 100$ Punkten und berichtet

- Trainingsgenauigkeit = 100%,
- Validierungsgenauigkeit = 60%.

- (a) Was ist die wahrscheinlichste Diagnose?
- (b) Was sollte als Nächstes probiert werden?

- ❗ (a) Klassische **Überanpassung**: bei Tiefe 20 und nur 100 Punkten schneidet der Baum um jeden Punkt ein winziges Gebiet aus. Jedes Blatt enthält einen Punkt, also 100% Trainingsgenauigkeit. Der Baum lernt Rauschen statt verallgemeinerbarer Struktur.
- (b) Die Tiefe reduzieren: kleinere Tiefen probieren (z. B. 2, 3, ..., 8) und diejenige mit der besten Validierungsgenauigkeit wählen.

9. Programmierübungen

Programmierübungen

Sie haben nun die Kenntnisse, die folgenden Übungen zu lösen:

`https://expert.ethz.ch/enrolled/SS26/mavt2/exercises`

- **Cancer Detection**
- **Gini Index**

Vergleich der Modelle

Eigenschaft	Lin. Reg.	Log. Reg.	Entsch.-Bäume	k-NN
Aufgabe	Regression	Klassifikation	Beides	Beides
Entscheidungsgrenze	Linear	Linear	Achsenparallel	Nicht-linear
Interpretierbarkeit	Hoch	Hoch	Hoch	Mittel
Geschw. (Training)	Schnell	Schnell	Schnell	Sofort
Geschw. (Vorhersage)	Schnell	Schnell	Schnell	Langsam
Nicht-lineare Daten	Nein	Nein	Ja	Ja
Kategoriale Merkmale	Kodierung nötig	Kodierung nötig	Direkt	Kodierung nötig
Ausreisserempfindlichkeit	Hoch (MSE)	Niedrig	Hoch	Hoch
Haupt-Hyperparameter	–	–	max_depth	k
Overfitting-Risiko	Niedrig	Niedrig	Hoch	Hoch

Kurzleitfaden: Lineare Modelle sind schnell und interpretierbar, aber auf einfache Grenzen beschränkt. Bäume erfassen Nichtlinearität, lernen aber Rauschen auswendig. k-NN ist flexibel, aber langsam und braucht sorgfältige Vorverarbeitung.